

Paper 03 — Quotes and Design Notes

Working notes. Copy-paste fodder for main.typ.

“As Nicola Jones reported in Nature in January 2025, LLMs still struggle to produce accurate citations, frequently returning wrong authors or fabricating papers that don’t exist. This happens because of an LLM’s architecture. LLMs are next-token prediction engines. They optimize for coherence and plausibility, not factual accuracy, with no internal mechanism to verify whether a journal exists or a DOI resolves to anything real. Lakera Research confirms that hallucination in citation generation is structurally linked to training data redundancy — highly cited papers get recalled with reasonable fidelity, while less prominent works are amalgamated into plausible-sounding fictions. The model doesn’t retrieve; it reconstructs. And in that reconstruction, lineage is lost.” [1]

“What emerges is something worse than missing provenance — it’s false provenance. GPTZero coined the term ‘vibe citing’ to describe how LLMs derive or combine real sources into uncanny imitations that appear accurate at first glance but collapse under verification.” [1]

Note: “Vibe citing” — a fabricated citation that feels right without being right — is worse than no citation at all, because it hijacks cognitive authority without earning it. RankeDB’s verification stage exists precisely to catch this: a claim without a verifiable source node in the Provenance DAG is not a fact, regardless of how plausibly the LLM presents it.

One-liner for §1: Vibe coding works because tests gate it. Vibe citing needs the same: an architectural verification gate between LLM output and the knowledge graph. RankeDB is that gate.

“Ghost references present with apparent legitimacy when indexed by systems like Google Scholar. This creates a feedback loop for LLMs, in which fabricated citations are discovered by other AI tools searching for verification in an already-polluted ecosystem. And without question, other AI systems will assume citations to be fact. When LLMs are trained on content that presents false citations, the lies are adopted wholesale, and continue to proliferate, emerging as false facts in AI-generated output. This is knowledge network decay, in action.” [1]

Note: The doom loop — fabricated citations propagate through the training-inference cycle, polluting future models — is a policy failure, not a technical one. Automated verification breaks the loop at ingestion: every claim’s citation is resolved by a verifier worker before the claim becomes a fact node. Pollution cannot propagate through an architecture that refuses to promote unverified claims into knowledge. RankeDB is that architecture by construction.

Design note (Talisman paraphrase, not a direct quote): the word “hallucination” misframes the problem. It suggests an occasional error mode — something that goes wrong sometimes — when in fact it is the normal operation of the model. Every LLM output is structurally the same thing: a plausible continuation of tokens. The model has no internal state that distinguishes “recalling a fact” from “inventing a fact.” Some continuations happen to correspond to reality; others don’t; the model cannot tell. This reframes RankeDB’s verification stage from “catching occasional errors” to “the architecture assumes every LLM output is unverified by default.” Verification is not defensive — it is baseline. The strong stance: you are not protecting against edge cases, you are protecting against the default behavior of the tool.

Design note — bounded verification + conviction + user confirmation:

Automatic verification has bounded reach. It works only within the domain of the graph — claims whose sources the DAG can already resolve (a DOI that exists as a source node, a fact extracted from a conversation that’s actually in the graph, an entity merge pointing to existing classification/entity nodes). Claims about the outside world that the graph has no source for cannot be auto-verified.

For the unverifiable case, RankeDB has two fallbacks:

1. *Conviction scoring — the claim enters the graph as a low-conviction node, marked as an uncorroborated LLM output. Not promoted to high-trust fact, but not discarded either.*
2. *User confirmation — for low-conviction thoughts that matter, the system asks the user to confirm. The confirmation becomes a new node with its own provenance (“User X confirmed this on date Y”). The user lends their cognitive authority, turning an uncorroborated claim into a user-verified one.*

The hierarchy:

- *Auto-verified (source exists in graph) → high conviction*
- *User-confirmed (low conviction + user authority) → high conviction with user provenance*
- *Uncorroborated → stays as low-conviction, queryable but explicitly marked*

Invariant: nothing is silently promoted from low conviction to high.

Design note — auth scoping (mention but defer):

Auth scoping is natively expressible in the RankeDB architecture: every node inherits its visibility from the visibility of its inputs, computed by DAG traversal (§3.4 of paper 1). This gives access control for free — no re-tagging, no re-indexing, no policy layer bolted on. Change visibility at a source, and the change propagates through every derivation automatically.

That covers the structural primitive. The policy layer is a much deeper problem:

- *Multi-user access models: groups, roles, hierarchies, delegation*
- *Project-level sharing: slicing the DAG without sharing the whole archive*
- *Temporal access: granted yesterday, revoked today — immutability + revocation is a hard combination*
- *Adversarial leakage: what can a denied user reconstruct from derived children?*
- *Compliance regimes: GDPR right-to-deletion vs. append-only; legal holds, retention, jurisdictional boundaries*
- *Shared-trust groups: when a team shares an archive, whose conviction counts for which facts?*

For paper 3: state that the mechanism exists, acknowledge the depth, and defer. Suggested framing:

“Auth scoping is natively expressible in the architecture: every node inherits its visibility from the visibility of its inputs, computed by DAG traversal. This gives RankeDB an access-control mechanism for free, without re-tagging or re-indexing. The full treatment — multi-user access models, project-level sharing, temporal revocation, compliance regimes, and adversarial leakage through derivations — is outside the scope of this paper and warrants separate research.”

The structural primitive is there; the policy layer isn’t. Being honest about that is a strength, not a weakness.

Note: RankeDB’s answer — citation verification agents as a mandatory stage in the worker chain. Every claim produced by an LLM worker must be counter-checked by a follow-up verifier before it becomes a fact node with full provenance. This is slower than trusting the LLM’s output directly. RankeDB accepts the latency cost under the bet that (a) model latency will keep improving and (b) correctness will win the race against speed-first architectures that skip verification.

Bibliography

- [1] J. Talisman, “Where Provenance Ends, Knowledge Decays.”